

MÓDULO 5: Traits y Genéricos (Sistema de Tipos)

Este módulo es la base de **todo** el ecosistema moderno de Rust. `axum`, `serde`, `tokio` — todos están contruidos sobre traits. Entender bien Static vs Dynamic Dispatch aquí te ahorrará muchísima confusión cuando veas `impl IntoResponse` o `Box<dyn Error>` más adelante.

5.1 Traits: Contratos de Comportamiento

Un trait define un conjunto de métodos que un tipo **promete implementar**. Es similar a una "interfaz" en otros lenguajes, pero más potente (puede tener implementaciones por defecto, genéricos, tipos asociados).

```
trait Resumible {
    fn resumen(&self) -> String;
    // Método con implementación por defecto: los tipos pueden
    usarla o sobreescribirla
    fn resumen_corto(&self) -> String {
        format!("(resumen: {})...",
            &self.resumen()[..10.min(self.resumen().len())])
    }
}

struct Artículo {
    titulo: String,
    contenido: String,
}

struct Tweet {
    usuario: String,
    texto: String,
}

impl Resumible for Artículo {
    fn resumen(&self) -> String {
        format!("{}", self.titulo, self.contenido)
    }
}

impl Resumible for Tweet {
    fn resumen(&self) -> String {
        format!("@{}: {}", self.usuario, self.texto)
    }
    // no sobreescribimos resumen_corto, usa la del default
}

fn main() {
    let art = Artículo {
        titulo: String::from("Rust 2.0"),
        contenido: String::from("Grandes novedades llegan este
```

```

año"),
    };
    let tw = Tweet {
        usuario: String::from("rustlang"),
        texto: String::from("Nuevo release disponible"),
    };
    println!("{}", art.resumen());
    println!("{}", tw.resumen_corto());
}

```

5.1.b Traits de Conversión Idiomáticos: From/Into, AsRef/AsMut y Blanket Implementations

Rust evita las conversiones mágicas e implícitas, proporcionando un conjunto de traits estándar para formalizar transformaciones de tipos sin costo oculto:

1. **From<T> / Into<U> (Conversiones con cambio de ownership)**: Si implementas From<T> for U, Rust implementa automáticamente Into<U> for T de forma gratuita vía una **Blanket Implementation** en la biblioteca estándar.
2. **AsRef<T> / AsMut<T> (Conversiones baratas por referencia)**: Permite que una función acepte cualquier tipo que pueda prestarse como una referencia inmutable (&T) o mutable (&mut T) sin clonar ni transferir ownership.

Diagrama Mental: Blanket Implementation de Into vía From

Tu código:

```

impl From<A> for B {
    fn from(a: A) -> B
}

```

Biblioteca Estándar (Blanket Impl):

```

impl<T, U> Into<U> for T where U: From<T> {
    fn into(self) -> U { U::from(self) }
}

```



¡Ganas `let b: B = a.into();` automáticamente!

Código Compilable

```
use std::path::Path;
```

```
// 1. From e Into
```

```
#[derive(Debug)]
```

```
struct IPv4(u32);
```

```
#[derive(Debug)]
```

```
struct SocketConfig {
```

```
    ip: IPv4,
```

```
    puerto: u16,
```

```
}
```

```
impl From<[u8; 4]> for IPv4 {
```

```
    fn from(octetos: [u8; 4]) -> Self {
```

```
        let ip_num = ((octetos[0] as u32) << 24)
```

```

        | ((octetos[1] as u32) << 16)
        | ((octetos[2] as u32) << 8)
        | (octetos[3] as u32);
    IPv4(ip_num)
}
}

// 2. AsRef para generalizar argumentos de paths/strings sin
allocations
fn leer_archivo_config<P: AsRef<Path>>(ruta: P) {
    let path_ref = ruta.as_ref();
    println!("Abriendo archivo en: {:?}", path_ref.display());
}

fn main() {
    // Conversión vía From
    let ip_directa = IPv4::from([127, 0, 0, 1]);

    // Conversión automática vía Into (gracias a la Blanket
    Implementation)
    let ip_into: IPv4 = [192, 168, 1, 1].into();
    println!("IPs: {:?} | {:?}", ip_directa, ip_into);

    // AsRef acepta &str, String o PathBuf indistintamente
    leer_archivo_config("config.toml");
    leer_archivo_config(String::from("/etc/app/config.toml"));
}

```

5.1.c La Regla del Huérfano (*Orphan Rule*) y Desacoplamiento con Newtypes

Para garantizar la coherencia global del sistema de tipos y evitar implementaciones duplicadas o ambiguas de un trait, Rust impone la **Regla del Huérfano**:

Regla de Coherencia: Solo puedes implementar un trait T para un tipo X si al menos **uno de los dos** (el trait T o el tipo X) está definido dentro de tu crate actual.

Si intentas implementar un trait externo sobre un tipo externo (por ejemplo, Display para Vec<T>), el compilador lo rechazará. La solución idiomática es el **Patrón Newtype**.

Código Compilable

```

use std::fmt;

// Intentar hacer `impl fmt::Display for Vec<String>` es ILEGAL
(ambos son externos).
// Solución: Envolver Vec<String> en un Newtype local.
struct ListaUsuarios(Vec<String>);

impl fmt::Display for ListaUsuarios {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "[{}]", self.0.join(", "))
    }
}

```

```
}

fn main() {
    let usuarios = ListaUsuarios(vec!["Alice".into(), "Bob".into(),
"Charlie".into()]);
    println!("Usuarios registrados: {}", usuarios); // Usa Display
del Newtype
}
```

⚠ Error típico del compilador: Violación de la Regla del Huérfano

```
use std::fmt;

// ✖ Error: Display es de std, y Vec<T> es de std
impl fmt::Display for Vec {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "Bytes: {:?}", self)
    }
}

error[E0117]: only traits defined in the current crate can be
implemented for types defined outside of the crate
--> src/main.rs:4:1
   |
4 | impl fmt::Display for Vec<u8> {
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^-----
   |                               |
   |                               `Vec` is not defined in the current crate
   | `Display` is not defined in the current crate
   |
   = note: define and implement a newtype struct instead
```

5.2 Traits Estándar Fundamentales

```
#[derive(Debug, Clone, Default)]
struct Config {
    nombre: String,
    nivel: u32,
}

fn main() {
    let c1 = Config { nombre: String::from("prod"), nivel: 3 };

    // Debug: {:?} – para desarrolladores, generado con #[derive]
    println!("{:?}", c1);
    println!("{:#?}", c1); // versión "pretty" multilínea

    // Clone: copia profunda explícita
    let c2 = c1.clone();
```

```

    // Default: valores "vacíos" razonables
    let c3 = Config::default(); // nombre: "", nivel: 0
    println!("{:?} {:?} {:?}", c1, c2, c3);
}

```

Para implementar Display (el trait de {}, para usuarios finales) **no existe #[derive]**, porque Rust no puede adivinar cómo quieres que se vea tu tipo para humanos — debes escribirlo tú mismo (como hicimos con ErrorApp en el Módulo 4).

```

use std::fmt;

impl fmt::Display for Config {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "Config[{}] nivel={}", self.nombre, self.nivel)
    }
}

```

♦ Tabla de traits estándar más usados

Trait	Habilita	Se deriva con
Debug	{:?} — impresión para debugging	#[derive(Debug)]
Clone	.clone() — copia profunda explícita	#[derive(Clone)]
Copy	copia implícita (solo tipos stack-only)	#[derive(Copy, Clone)]
Default	Tipo::default()	#[derive(Default)]
PartialEq	==, !=	#[derive(PartialEq)]
PartialOrd	<, >, .sort()	#[derive(PartialOrd)]
Display	{ } — impresión para usuarios	manual (no derivable)
Hash	usar como clave de HashMap	#[derive(Hash)]

5.3 Genéricos: Código que Funciona sobre Cualquier Tipo

```

// Función genérica: T puede ser cualquier tipo que implemente
PartialOrd
fn mayor<T: PartialOrd>(lista: &[T]) -> &T {
    let mut mayor = &lista[0];
    for item in lista {
        if item > mayor {
            mayor = item;
        }
    }
    mayor
}

// Struct genérico
struct Par<T> {
    primero: T,
    segundo: T,
}

```

```
impl<T: std::fmt::Display + PartialOrd> Par<T> {
    fn mostrar_mayor(&self) {
        if self.primerio >= self.segundo {
            println!("El mayor es: {}", self.primerio);
        } else {
            println!("El mayor es: {}", self.segundo);
        }
    }
}

fn main() {
    let numeros = vec![34, 50, 25, 100, 65];
    println!("El mayor número es: {}", mayor(&numeros));

    let palabras = vec!["manzana", "zebra", "banana"];
    println!("La mayor palabra es: {}", mayor(&palabras));

    let par = Par { primero: 5, segundo: 10 };
    par.mostrar_mayor();
}
```

⚠ Error típico del compilador #1: Genérico sin trait bound necesario

```
fn mayor<T>(lista: &[T]) -> &T {
    let mut mayor = &lista[0];
    for item in lista {
        if item > mayor { // ❌ T no garantiza soportar >
            mayor = item;
        }
    }
    mayor
}
```

error[E0369]: binary operation `>` cannot be applied to type `T`
--> src/main.rs:4:18

```
4 |         if item > mayor {
      |             ^      ----- T
      |             |
      |             T
```

help: consider restricting type parameter `T`

```
1 | fn mayor<T: std::cmp::PartialOrd>(lista: &[T]) -> &T {
    ++++++
```

La lección: genéricos en Rust NO son "cualquier tipo, y ya veremos en runtime si funciona" (como los genéricos con type erasure en Java antes de generics reales, o duck typing en

Python). Son **contratos verificados en compile-time**: si tu función usa `>`, el compilador exige que declares explícitamente que `T` soporta esa operación vía `PartialOrd`. Cero sorpresas en runtime.

5.4 Trait Bounds y where clauses

```
use std::fmt::{Debug, Display};

// Sintaxis compacta (bien para 1-2 bounds)
fn imprimir<T: Display + Debug>(item: T) {
    println!("{}", item);
}

// Sintaxis where (mejor legibilidad con múltiples bounds/genéricos)
fn procesar<T, U>(a: T, b: U) -> String
where
    T: Display + Clone,
    U: Debug + Default,
{
    format!("{}", a, b)
}

fn main() {
    imprimir(42);
    println!("{}", procesar("hola", 5));
}
```

 **Regla práctica:** con 1 genérico y ≤ 2 bounds, usa la sintaxis inline. Con 2+ genéricos o bounds complejos, usa where — es mucho más legible y es el estándar en código de producción (lo verás constantemente en firmas de axum y tokio).

5.5 impl Trait: Sintaxis Simplificada

```
// Como parámetro: equivalente a un genérico con bound, más legible
fn notificar(item: &impl Resumible) {
    println!("{}", item.resumen());
}

// Como retorno: útil para devolver iteradores o closures sin
nombrar el tipo exacto
fn crear_articulo() -> impl Resumible {
    Artículo {
        titulo: String::from("Nuevo"),
        contenido: String::from("Contenido generado"),
    }
}

fn main() {
    let art = crear_articulo();
    notificar(&art);
}
```

```
}
```

impl Trait en el retorno es especialmente valioso porque **algunos tipos no se pueden nombrar** (como los que genera un closure o un iterador con `.map().filter()` encadenados) — sin impl Trait tendrías que usar `Box<dyn Trait>` innecesariamente, con costo de heap allocation.

5.6 Dispatch Estático vs. Dinámico — El Concepto Más Importante del Módulo

Esta es la decisión de diseño que **más impacta el rendimiento y la arquitectura** de tu código Rust.

◆ Dispatch Estático (impl Trait / genéricos) — Monomorfización

```
fn notificar_estatico<T: Resumible>(item: &T) {
    println!("{}", item.resumen());
}

fn main() {
    let art = Articulo { titulo: String::from("A"), contenido:
String::from("B") };
    let tw = Tweet { usuario: String::from("C"), texto:
String::from("D") };
    notificar_estatico(&art);
    notificar_estatico(&tw);
}
```

Qué hace el compilador: genera, en tiempo de compilación, una copia especializada de `notificar_estatico` para cada tipo concreto usado — esto se llama **monomorfización**:

```
// El compilador genera INTERNAMENTE algo equivalente a:
fn notificar_estatico_articulo(item: &Articulo) { println!("{}",
item.resumen()); }
fn notificar_estatico_tweet(item: &Tweet) { println!("{}",
item.resumen()); }
```

🧠 Diagrama mental: Dispatch Estático

Compile-time:		Runtime:
<code>notificar_estatico<Articulo></code>	→	llamada DIRECTA, sin indirección
<code>notificar_estatico<Tweet></code>	→	llamada DIRECTA, sin indirección

Ventaja: velocidad máxima (el compilador puede hasta inlinear la llamada)

Costo: tamaño del binario crece (una copia por tipo usado) — "code bloat"

◆ Dispatch Dinámico (dyn Trait) — Vtables

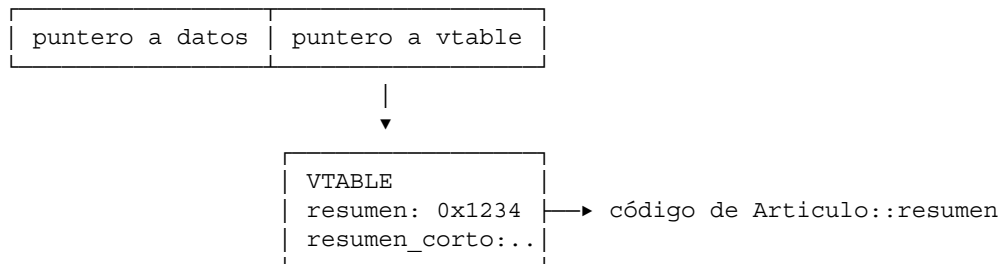
```
fn notificar_dinamico(item: &dyn Resumible) {
    println!("{}", item.resumen());
}

fn main() {
    let art = Artículo { titulo: String::from("A"), contenido:
String::from("B") };
    let tw = Tweet { usuario: String::from("C"), texto:
String::from("D") };

    // Podemos meter tipos DISTINTOS en la misma colección, algo
imposible con genéricos
    let items: Vec<Box<dyn Resumible>> = vec![Box::new(art),
Box::new(tw)];
    for item in &items {
        notificar_dinamico(item.as_ref());
    }
}
```

🧠 Diagrama mental: Dispatch Dinámico

`&dyn Resumible` es un "fat pointer" (puntero gordo), ocupa 2 palabras de máquina:



Runtime: la llamada pasa por la vtable (una indirección extra) para saber

QUÉ implementación ejecutar, porque el tipo concreto ya no se conoce en compile-time – solo se sabe "algo que implementa Resumible".

Ventaja: un solo binario compilado, colecciones heterogéneas, menor tamaño de binario

Costo: pequeño overhead de indirección en cada llamada, y no se puede inlinear

♦ Tabla de Decisión: ¿Cuándo usar cada uno?

Situación	Usa
Sabes el tipo concreto en compile-time, priorizas velocidad máxima	Dispatch estático (impl Trait, genéricos <T: Trait>)
Necesitas una colección con tipos DISTINTOS que comparten un trait	Dispatch dinámico (Vec<Box<dyn Trait>>)
Estás escribiendo una librería y quieres	Dispatch dinámico

Situación	Usa
binarios más pequeños para tus usuarios	
El trait se usa en "hot path" (loop crítico de rendimiento)	Dispatch estático
Necesitas guardar "callbacks" o "handlers" de tipos variados (ej: middlewares en axum)	Dispatch dinámico

⚠ Error típico del compilador #2: Trait objects y "object safety"

```
trait Clonable {
    fn clonar(&self) -> Self; // Self como tipo de retorno
}

fn usar(item: &dyn Clonable) {} // ❌

error[E0038]: the trait `Clonable` cannot be made into an object
--> src/main.rs:4:19
   |
4  | fn usar(item: &dyn Clonable) {}
   |               ^^^^^^^^^^^ `Clonable` cannot be made into an
object
   |
   = note: method `clonar` references the `Self` type in its return
type
```

Por qué ocurre: una vtable necesita punteros a funciones con firma *fija* conocida en compile-time. Si un método devuelve Self, el tamaño de retorno **depende del tipo concreto**, que es justo lo que dyn Trait decide no conocer. Esto se llama **"object safety"**: un trait solo puede convertirse en dyn Trait si ninguno de sus métodos usa Self por valor (como retorno o parámetro) ni genéricos propios. Clone es el ejemplo clásico de trait que **no es object-safe** por esta razón exacta — nunca verás dyn Clone.

5.6.b El Trait Sized, ?Sized y Estructura de Fat Pointers

En Rust, casi todos los tipos implementan automáticamente el marker trait Sized, lo que indica que su tamaño en bytes es constante y conocido en tiempo de compilación.

- **DSTs (Dynamically Sized Types):** Tipos como str, [T] o dyn Trait no tienen un tamaño conocido en compile-time (son !Sized).
- **Bound ?Sized (Relaxed Bound):** Por defecto, toda función genérica fn func<T>(x: T) asume implícitamente T: Sized. Si necesitas que acepte tanto tipos de tamaño fijo como dinámicos por referencia, debes relajar el bound con T: ?Sized.

🧠 Diagrama Mental: Fat Pointers (&dyn Trait y &[T])

Un puntero normal ocupa **1 palabra de máquina** (8 bytes en 64-bit). Un *Fat Pointer* a un

tipo !Sized ocupa **2 palabras de máquina**:

Puntero Sized (&i32):

Ptr a Memoria	(8 bytes)
---------------	-----------

Fat Pointer a Slice (&[u8]):

Ptr a Datos	Longitud (N)	(16 bytes)
-------------	--------------	------------

Fat Pointer a Trait Object (&dyn Trait):

Ptr a Datos	Ptr a Vtable	(16 bytes)
-------------	--------------	------------

Código Compilable

```
// T: ?Sized permite aceptar tanto `String` como `str` detrás de una
referencia
fn imprimir_informe<T: std::fmt::Display + ?Sized>(dato: &T) {
    println!("INFORME: {}", dato);
}

fn main() {
    let texto_stack: &str = "cadena_slice";          // str es !Sized
    let texto_heap: String = "cadena_heap".into();    // String es Sized

    imprimir_informe(texto_stack);
    imprimir_informe(&texto_heap);
    // Inspección de tamaño de fat pointers vs thin pointers
    println!("Tamaño de &i32: {} bytes", std::mem::size_of::<&i32>());
    println!("Tamaño de &dyn std::fmt::Display: {} bytes",
std::mem::size_of::<&dyn std::fmt::Display>());
    println!("Tamaño de &[u8]: {} bytes", std::mem::size_of::<&[u8]>());
}
```

5.7 Traits con Tipos Asociados

```
trait Contenedor {
    type Item; // tipo asociado: cada implementación define CUÁL es
    fn obtener(&self, indice: usize) -> Option<&Self::Item>;
}

struct Pila<T> {
    elementos: Vec<T>,
}

impl<T> Contenedor for Pila<T> {
    type Item = T;
    fn obtener(&self, indice: usize) -> Option<&T> {
        self.elementos.get(indice)
    }
}

fn main() {
    let pila = Pila { elementos: vec![1, 2, 3] };
}
```

```
println!("{:?}", pila.obtener(1));
}
```

Diferencia con genéricos: con tipos asociados, cada tipo (`Pila<T>`) tiene **una sola** implementación de `Contenedor` (con `Item` fijo). Con genéricos (`trait Contenedor<Item>`), un mismo tipo podría implementar el `trait` múltiples veces para distintos `Item`. Usa tipos asociados cuando la relación es "uno a uno" (como en `Iterator`, donde `Item` es único por iterador).

5.7.b Supertraits y Sobrecarga de Operadores con `std::ops`

Supertraits (Herencia de Requisitos de Traits)

Un `trait` puede depender de otro mediante la sintaxis `trait SubTrait: SuperTrait`. Cualquier tipo que desee implementar `SubTrait` está obligado por el compilador a implementar también `SuperTrait`.

Sobrecarga de Operadores (`std::ops`)

Rust no permite crear operadores nuevos, pero permite sobrecargar los existentes (+, -, *, /, ==) implementando los `traits` correspondientes del módulo `std::ops`.

Código Compilable

```
use std::fmt::Display;
use std::ops::Add;

// 1. Supertrait: Logueable EXIGE que el tipo también implemente
// Display
trait Logueable: Display {
    fn log_consola(&self) {
        println!("[LOG DEL SISTEMA] {}", self);
    }
}

// 2. Sobrecarga de Operadores
#[derive(Debug, Clone, Copy, PartialEq)]
struct Vector2D {
    x: f64,
    y: f64,
}

impl Display for Vector2D {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) ->
    std::fmt::Result {
        write!(f, "Vec2D({}, {})", self.x, self.y)
    }
}
```

```

    }
}

// Implementación del Supertrait (válida porque Vector2D implementa
Display)
impl Logueable for Vector2D {}

// Sobrecarga del operador `+`
impl Add for Vector2D {
    type Output = Self;

    fn add(self, rhs: Self) -> Self::Output {
        Vector2D {
            x: self.x + rhs.x,
            y: self.y + rhs.y,
        }
    }
}

fn main() {
    let v1 = Vector2D { x: 10.0, y: 5.0 };
    let v2 = Vector2D { x: 3.0, y: 2.0 };

    // Uso del operador sobrecargado `+`
    let v3 = v1 + v2;

    // Uso del método derivado del supertrait
    v3.log_consola(); // [LOG DEL SISTEMA] Vec2D(13, 7)
}

```

Ejercicio Integrador — Módulo 5

Proyecto: Sistema de Notificaciones Multi-Canal

Crea cargo new sistema_notificaciones:

1. Define un trait Notificador con:

```
trait Notificador {  
    fn enviar(&self, mensaje: &str) -> Result<(), String>;  
    fn nombre_canal(&self) -> String {  
        String::from("Canal genérico") // default  
    }  
}
```
2. Implementa Notificador para tres structs: Email { direccion: String }, SMS { numero: String }, Slack { canal: String }. Cada uno simula el "envío" imprimiendo un mensaje distinto y devuelve Err si el mensaje está vacío.
3. Escribe una función con **dispatch estático**: fn notificar_uno<T: Notificador>(n: &T, msg: &str).
4. Escribe una función con **dispatch dinámico**: fn notificar_todos(notificadores: &[Box<dyn Notificador>], msg: &str) que itere y llame a enviar en cada uno, manejando el Result de cada llamada.
5. En main:
 - Crea un Vec<Box<dyn Notificador>> con instancias de los 3 tipos.
 - Llama a notificar_todos.
 - Provoca a propósito el error E0038 intentando meter un trait con un método que devuelva Self dentro de un dyn — pégame el error si quieres que lo analicemos juntos.
6. **Genéricos con where**: escribe fn resumen_multiple<T>(items: &[T]) -> String where T: Notificador que concatene nombre_canal() de todos con ", " usando .join().

Ejercicio Integrador 2

Proyecto: Motor de Transformación y Aritmética de Métricas Tipadas

Crea cargo new motor_metricas e implementa:

1. **Newtypes y Regla del Huérfano**:
 - Modela struct MetricaWatts(f64) y struct MetricaAmperios(f64).
 - Implementa Display para ambos usando el patrón newtype.
2. **Conversiones Idiomáticas**:
 - Implementa From<f64> para MetricaWatts.
 - Implementa AsRef<f64> para permitir inspección directa sin mover el valor.
3. **Sobrecarga de Operadores con std::ops::Add**:
 - Implementa la suma entre dos MetricaWatts retornando un nuevo MetricaWatts.
4. **Colecciones Heterogéneas y Fat Pointers**:
 - Define el supertrait trait SensorMedicion: Display con un método fn valor_normalizado(&self) -> f64.
 - Implementa SensorMedicion para tus structs.
 - Crea un vector heterogéneo Vec<Box<dyn SensorMedicion>> (verificando el tamaño de los fat pointers en memoria) e itera sobre ellos aplicando monomorfización y despacho dinámico según corresponda.